

OpenStack & RHOSO End-to-End Deployment Guide

Complete deployment guide for **Red Hat OpenStack Services on OpenShift (RHOSO)** and **Classic OpenStack (DevStack / Packstack / Manual)**

Covers: **Ubuntu 22.04/24.04** and **RHEL 8/9**

Table of Contents

1. [Architecture Overview](#)
 2. [Prerequisites](#)
 3. [Part 1 – Classic OpenStack Deployment](#)
 - [Ubuntu: DevStack \(All-in-One\)](#)
 - [Ubuntu: Manual \(Keystone → Nova → Neutron → Glance → Cinder → Horizon\)](#)
 - [RHEL: Packstack \(All-in-One\)](#)
 - [RHEL: TripleO / Director Deployment](#)
 4. [Part 2 – RHOSO \(OpenStack on OpenShift\)](#)
 - [Prerequisites & OpenShift Cluster](#)
 - [Install OpenStack Operators](#)
 - [Deploy OpenStack Control Plane](#)
 - [Deploy OpenStack Data Plane \(Compute Nodes\)](#)
 - [Post-Deployment Validation](#)
 5. [Common Post-Deployment Tasks](#)
 6. [Troubleshooting](#)
 7. [GitHub Repository Structure](#)
-

Architecture Overview

Classic OpenStack

Controller Node
Keystone Glance Nova-API Neutron Horizon
Compute Node(s)
Nova-Compute Neutron-Agent Libvirt
Storage Node (optional)
Cinder Swift Ceph

RHOSO (OpenStack on OpenShift)

OpenShift Cluster			
<table><tr><td>openstack-operators (OLM)</td></tr><tr><td>openstack-galera rabbitmq memcached</td></tr><tr><td>keystone glance nova neutron cinder</td></tr></table>	openstack-operators (OLM)	openstack-galera rabbitmq memcached	keystone glance nova neutron cinder
openstack-operators (OLM)			
openstack-galera rabbitmq memcached			
keystone glance nova neutron cinder			
EDPM (External Data Plane Management)			
Bare-metal or VM compute nodes			
managed via ansible / dataplane-operator			

Prerequisites

Hardware / VM Sizing

Role	vCPUs	RAM	Disk
All-in-One (DevStack/Packstack)	4+	16 GB	100 GB
RHOSO OCP Control Plane (per master)	8	32 GB	120 GB
RHOSO Worker / Compute Node	4	16 GB	100 GB

Network Requirements

- Management network (SSH, API): e.g. 192.168.1.0/24
- Provider / External network: e.g. 10.0.0.0/24

- Internal/tenant network (VXLAN/VLAN)
 - Storage network (optional): e.g. 172.16.0.0/24
-

Part 1 – Classic OpenStack Deployment

Ubuntu: DevStack (All-in-One)

Suitable for **development / testing**. Not for production.

Step 1 – System Preparation

```
# Update system
sudo apt-get update -y && sudo apt-get upgrade -y

# Install dependencies
sudo apt-get install -y git python3-pip curl wget vim net-tools

# Disable firewall (dev only)
sudo ufw disable

# Set hostname
sudo hostnamectl set-hostname controller
echo "127.0.1.1 controller" | sudo tee -a /etc/hosts
```

Step 2 – Create Stack User

```
sudo useradd -s /bin/bash -d /opt/stack -m stack
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
sudo chmod 0440 /etc/sudoers.d/stack
sudo su - stack
```

Step 3 – Clone DevStack

```
git clone https://opendev.org/openstack/devstack /opt/stack/devstack
cd /opt/stack/devstack
```

Step 4 – Create local.conf

```
cat > /opt/stack/devstack/local.conf << 'EOF'
[[local|localrc]]
# Passwords
ADMIN_PASSWORD=Admin1234!
DATABASE_PASSWORD=Admin1234!
RABBIT_PASSWORD=Admin1234!
SERVICE_PASSWORD=Admin1234!

# Host IP (replace with your actual IP)
HOST_IP=192.168.1.100

# Branch
GIT_BASE=https://opendev.org
OPENSTACK_RELEASES=2024.1

# Services to enable
ENABLED_SERVICES=rabbit,mysql,key
ENABLED_SERVICES+=,n-api,n-cpu,n-cond,n-sch,n-novnc,n-api-meta,placement-api
ENABLED_SERVICES+=,g-api,g-reg
ENABLED_SERVICES+=,q-svc,q-agt,q-dhcp,q-l3,q-meta,neutron
ENABLED_SERVICES+=,cinder,c-api,c-vol,c-sch
ENABLED_SERVICES+=,horizon

# Neutron ML2 Config
Q_PLUGIN=ml2
Q_AGENT=openvswitch
ENABLE_TENANT_VLANS=True

# Logging
LOGFILE=/opt/stack/logs/stack.sh.log
LOGDAYS=2

# Image
IMAGE_URLS="https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-disk.img"
EOF
```

Step 5 – Run DevStack

```
cd /opt/stack/devstack
./stack.sh
```



Takes 20–45 minutes. Horizon available at: http://<HOST_IP>/dashboard

Step 6 – Verify Deployment

```
source /opt/stack/devstack/openrc admin admin
```

```
openstack service list
openstack endpoint list
openstack network list
openstack image list
openstack compute service list
```

Ubuntu: Manual Multi-Service

Production-ready step-by-step for **Ubuntu 22.04 LTS**

Assumes: controller = 192.168.1.10 , compute = 192.168.1.11

1. All Nodes – Base Setup

```
# On ALL nodes
sudo apt-get update -y && sudo apt-get upgrade -y
sudo apt-get install -y software-properties-common python3-openstackclient

# Add OpenStack Caracal (2024.1) repo
sudo add-apt-repository cloud-archive:caracal -y
sudo apt-get update -y && sudo apt-get dist-upgrade -y

# Set NTP
sudo apt-get install -y chrony
sudo systemctl enable --now chrony
```

2. Controller – Database & Message Queue

```
# MariaDB
sudo apt-get install -y mariadb-server python3-pymysql

sudo tee /etc/mysql/mariadb.conf.d/99-openstack.cnf << 'EOF'
[mysqld]
bind-address = 192.168.1.10
default-storage-engine = innodb
innodb_file_per_table = on
max_connections = 4096
collation-server = utf8_general_ci
character-set-server = utf8
EOF
```

```

sudo systemctl restart mariadb
sudo mysql_secure_installation

# RabbitMQ
sudo apt-get install -y rabbitmq-server
sudo rabbitmqctl add_user openstack RabbitPass123
sudo rabbitmqctl set_permissions openstack ".*" ".*" ".*"

# Memcached
sudo apt-get install -y memcached python3-memcache
sudo sed -i 's/127.0.0.1/192.168.1.10/' /etc/memcached.conf
sudo systemctl restart memcached

# Etcd
sudo apt-get install -y etcd
sudo tee /etc/default/etcd << 'EOF'
ETCD_NAME="controller"
ETCD_DATA_DIR="/var/lib/etcd"
ETCD_INITIAL_CLUSTER_STATE="new"
ETCD_INITIAL_CLUSTER_TOKEN="etcd-cluster-01"
ETCD_INITIAL_CLUSTER="controller=http://192.168.1.10:2380"
ETCD_INITIAL_ADVERTISE_PEER_URLS="http://192.168.1.10:2380"
ETCD_ADVERTISE_CLIENT_URLS="http://192.168.1.10:2379"
ETCD_LISTEN_PEER_URLS="http://0.0.0.0:2380"
ETCD_LISTEN_CLIENT_URLS="http://192.168.1.10:2379"
EOF
sudo systemctl enable --now etcd

```

3. Controller – Keystone (Identity Service)

```

# Create database
sudo mysql -u root -p << 'EOF'
CREATE DATABASE keystone;
GRANT ALL PRIVILEGES ON keystone.* TO 'keystone'@'localhost' IDENTIFIED BY 'Keyst
GRANT ALL PRIVILEGES ON keystone.* TO 'keystone'@'%' IDENTIFIED BY 'KeystoneDB123
FLUSH PRIVILEGES;
EOF

# Install Keystone
sudo apt-get install -y keystone

# Configure /etc/keystone/keystone.conf
sudo tee -a /etc/keystone/keystone.conf << 'EOF'
[database]
connection = mysql+pymysql://keystone:KeystoneDB123@192.168.1.10/keystone

```

```

[token]
provider = fernet
EOF

# Populate database & bootstrap
sudo -H -u keystone keystone-manage db_sync
sudo keystone-manage fernet_setup --keystone-user keystone --keystone-group keyst
sudo keystone-manage credential_setup --keystone-user keystone --keystone-group k

sudo keystone-manage bootstrap --bootstrap-password Admin1234! \
  --bootstrap-admin-url http://192.168.1.10:5000/v3/ \
  --bootstrap-internal-url http://192.168.1.10:5000/v3/ \
  --bootstrap-public-url http://192.168.1.10:5000/v3/ \
  --bootstrap-region-id RegionOne

# Configure Apache
sudo tee /etc/apache2/sites-available/keystone.conf << 'EOF'
Listen 5000
<VirtualHost *:5000>
    WSGIDaemonProcess keystone-public processes=5 threads=1 user=keystone group=k
    WSGIProcessGroup keystone-public
    WSGIScriptAlias / /usr/bin/keystone-wsgi-public
    WSGIApplicationGroup %{GLOBAL}
    <IfVersion >= 2.4>
        ErrorLogFormat "%{cu}t %M"
    </IfVersion>
    ErrorLog /var/log/apache2/keystone.log
    CustomLog /var/log/apache2/keystone_access.log combined
</VirtualHost>
EOF

sudo a2ensite keystone && sudo systemctl restart apache2

# Create admin openrc
cat > ~/admin-openrc.sh << 'EOF'
export OS_USERNAME=admin
export OS_PASSWORD=Admin1234!
export OS_PROJECT_NAME=admin
export OS_USER_DOMAIN_NAME=Default
export OS_PROJECT_DOMAIN_NAME=Default
export OS_AUTH_URL=http://192.168.1.10:5000/v3
export OS_IDENTITY_API_VERSION=3
export OS_IMAGE_API_VERSION=2
EOF

source ~/admin-openrc.sh

```

```
# Create service project and roles
openstack project create --domain default --description "Service Project" service
openstack role create member
openstack role create reader
```

4. Controller – Glance (Image Service)

```
sudo mysql -u root -p << 'EOF'
CREATE DATABASE glance;
GRANT ALL PRIVILEGES ON glance.* TO 'glance'@'localhost' IDENTIFIED BY 'GlanceDB123';
GRANT ALL PRIVILEGES ON glance.* TO 'glance'@'%' IDENTIFIED BY 'GlanceDB123';
EOF
```

```
source ~/admin-openrc.sh
openstack user create --domain default --password Glance123! glance
openstack role add --project service --user glance admin
openstack service create --name glance --description "OpenStack Image" image
openstack endpoint create --region RegionOne image public http://192.168.1.10:929
openstack endpoint create --region RegionOne image internal http://192.168.1.10:9
openstack endpoint create --region RegionOne image admin http://192.168.1.10:9292
```

```
sudo apt-get install -y glance
```

```
sudo tee /etc/glance/glance-api.conf << 'EOF'
[DEFAULT]
[database]
connection = mysql+pymysql://glance:GlanceDB123@192.168.1.10/glance
[keystone_authtoken]
www_authenticate_uri = http://192.168.1.10:5000
auth_url = http://192.168.1.10:5000
memcached_servers = 192.168.1.10:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = glance
password = Glance123!
[paste_deploy]
flavor = keystone
[glance_store]
stores = file,http
default_store = file
filesystem_store_datadir = /var/lib/glance/images/
EOF
```

```

sudo -H -u glance glance-manage db_sync
sudo systemctl enable --now glance-api

# Upload Cirros test image
wget https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-disk.img
glance image-create --name "cirros" \
  --file cirros-0.6.2-x86_64-disk.img \
  --disk-format qcow2 \
  --container-format bare \
  --visibility public

```

5. Controller – Nova (Compute Service)

```

sudo mysql -u root -p << 'EOF'
CREATE DATABASE nova_api;
CREATE DATABASE nova;
CREATE DATABASE nova_cell0;
GRANT ALL PRIVILEGES ON nova_api.* TO 'nova'@'localhost' IDENTIFIED BY 'NovaDB123';
GRANT ALL PRIVILEGES ON nova_api.* TO 'nova'@'%' IDENTIFIED BY 'NovaDB123';
GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'localhost' IDENTIFIED BY 'NovaDB123';
GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'%' IDENTIFIED BY 'NovaDB123';
GRANT ALL PRIVILEGES ON nova_cell0.* TO 'nova'@'localhost' IDENTIFIED BY 'NovaDB123';
GRANT ALL PRIVILEGES ON nova_cell0.* TO 'nova'@'%' IDENTIFIED BY 'NovaDB123';
EOF

source ~/admin-openrc.sh
openstack user create --domain default --password Nova123! nova
openstack role add --project service --user nova admin
openstack service create --name nova --description "OpenStack Compute" compute
openstack endpoint create --region RegionOne compute public http://192.168.1.10:87
openstack endpoint create --region RegionOne compute internal http://192.168.1.10:87
openstack endpoint create --region RegionOne compute admin http://192.168.1.10:87

# Placement
openstack user create --domain default --password Placement123! placement
openstack role add --project service --user placement admin
openstack service create --name placement --description "Placement API" placement
openstack endpoint create --region RegionOne placement public http://192.168.1.10:87
openstack endpoint create --region RegionOne placement internal http://192.168.1.10:87
openstack endpoint create --region RegionOne placement admin http://192.168.1.10:87

sudo apt-get install -y nova-api nova-conductor nova-novncproxy nova-scheduler pl

# Edit /etc/nova/nova.conf (key sections shown)

```

```

sudo tee -a /etc/nova/nova.conf << 'EOF'
[DEFAULT]
my_ip = 192.168.1.10
transport_url = rabbit://openstack:RabbitPass123@192.168.1.10:5672/
[api_database]
connection = mysql+pymysql://nova:NovaDB123@192.168.1.10/nova_api
[database]
connection = mysql+pymysql://nova:NovaDB123@192.168.1.10/nova
[api]
auth_strategy = keystone
[keystone_authtoken]
www_authenticate_uri = http://192.168.1.10:5000/
auth_url = http://192.168.1.10:5000/
memcached_servers = 192.168.1.10:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = nova
password = Nova123!
[vnc]
enabled = true
server_listen = $my_ip
server_proxyclient_address = $my_ip
[glance]
api_servers = http://192.168.1.10:9292
[oslo_concurrency]
lock_path = /var/lib/nova/tmp
[placement]
region_name = RegionOne
project_domain_name = Default
project_name = service
auth_type = password
user_domain_name = Default
auth_url = http://192.168.1.10:5000/v3
username = placement
password = Placement123!
EOF

sudo -H -u nova nova-manage api_db sync
sudo -H -u nova nova-manage cell_v2 map_cell0
sudo -H -u nova nova-manage cell_v2 create_cell --name=cell1 --verbose
sudo -H -u nova nova-manage db sync

sudo systemctl enable --now nova-api nova-scheduler nova-conductor nova-novncprox

```

6. Controller – Neutron (Networking Service)

```
sudo mysql -u root -p << 'EOF'
CREATE DATABASE neutron;
GRANT ALL PRIVILEGES ON neutron.* TO 'neutron'@'localhost' IDENTIFIED BY 'Neutron'
GRANT ALL PRIVILEGES ON neutron.* TO 'neutron'@'%' IDENTIFIED BY 'NeutronDB123';
EOF

source ~/admin-openrc.sh
openstack user create --domain default --password Neutron123! neutron
openstack role add --project service --user neutron admin
openstack service create --name neutron --description "OpenStack Networking" netw
openstack endpoint create --region RegionOne network public http://192.168.1.10:9
openstack endpoint create --region RegionOne network internal http://192.168.1.10
openstack endpoint create --region RegionOne network admin http://192.168.1.10:96

sudo apt-get install -y neutron-server neutron-plugin-ml2 \
    neutron-openvswitch-agent neutron-l3-agent neutron-dhcp-agent \
    neutron-metadata-agent

# /etc/neutron/neutron.conf (key sections)
sudo tee -a /etc/neutron/neutron.conf << 'EOF'
[DEFAULT]
core_plugin = ml2
service_plugins = router
transport_url = rabbit://openstack:RabbitPass123@192.168.1.10
auth_strategy = keystone
notify_nova_on_port_status_changes = true
notify_nova_on_port_data_changes = true
[keystone_authtoken]
www_authenticate_uri = http://192.168.1.10:5000
auth_url = http://192.168.1.10:5000
memcached_servers = 192.168.1.10:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = neutron
password = Neutron123!
[database]
connection = mysql+pymysql://neutron:NeutronDB123@192.168.1.10/neutron
[nova]
auth_url = http://192.168.1.10:5000
auth_type = password
project_domain_name = Default
user_domain_name = Default
```

```

region_name = RegionOne
project_name = service
username = nova
password = Noval23!
EOF

sudo tee /etc/neutron/plugins/ml2/ml2_conf.ini << 'EOF'
[ml2]
type_drivers = flat,vlan,vxlan
tenant_network_types = vxlan
mechanism_drivers = openvswitch,l2population
extension_drivers = port_security

[ml2_type_flat]
flat_networks = provider

[ml2_type_vxlan]
vni_ranges = 1:1000

[securitygroup]
enable_ipset = true
EOF

sudo tee /etc/neutron/plugins/ml2/openvswitch_agent.ini << 'EOF'
[ovs]
bridge_mappings = provider:br-provider
local_ip = 192.168.1.10

[agent]
tunnel_types = vxlan
l2_population = true

[securitygroup]
firewall_driver = iptables_hybrid
EOF

sudo -H -u neutron neutron-db-manage \
    --config-file /etc/neutron/neutron.conf \
    --config-file /etc/neutron/plugins/ml2/ml2_conf.ini upgrade head

sudo systemctl enable --now neutron-server neutron-openvswitch-agent \
    neutron-dhcp-agent neutron-metadata-agent neutron-l3-agent

```

7. Controller – Horizon (Dashboard)

```

sudo apt-get install -y openstack-dashboard

sudo sed -i \
"s/OPENSTACK_HOST = \"127.0.0.1\"/OPENSTACK_HOST = \"192.168.1.10\"/" \
/etc/openstack-dashboard/local_settings.py

sudo sed -i \
"s/#OPENSTACK_KEYSTONE_URL/OPENSTACK_KEYSTONE_URL/" \
/etc/openstack-dashboard/local_settings.py

sudo systemctl restart apache2
# Access: http://192.168.1.10/horizon

```

8. Compute Node Setup (Ubuntu)

```

# On compute node (192.168.1.11)
sudo apt-get install -y nova-compute neutron-openvswitch-agent

# Configure /etc/nova/nova.conf on compute
sudo tee -a /etc/nova/nova.conf << 'EOF'
[DEFAULT]
my_ip = 192.168.1.11
transport_url = rabbit://openstack:RabbitPass123@192.168.1.10
[api]
auth_strategy = keystone
[keystone_authtoken]
www_authenticate_uri = http://192.168.1.10:5000/
auth_url = http://192.168.1.10:5000/
memcached_servers = 192.168.1.10:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = nova
password = Nova123!
[vnc]
enabled = true
server_listen = 0.0.0.0
server_proxyclient_address = $my_ip
novncproxy_base_url = http://192.168.1.10:6080/vnc_lite.html
[glance]
api_servers = http://192.168.1.10:9292
[oslo_concurrency]
lock_path = /var/lib/nova/tmp
[placement]

```

```
region_name = RegionOne
project_domain_name = Default
project_name = service
auth_type = password
user_domain_name = Default
auth_url = http://192.168.1.10:5000/v3
username = placement
password = Placement123!
EOF

sudo systemctl enable --now nova-compute

# Discover compute host on controller
# (run on controller)
source ~/admin-openrc.sh
sudo -H -u nova nova-manage cell_v2 discover_hosts --verbose
```

RHEL: Packstack (All-in-One)

Works on **RHEL 8/9** and **CentOS Stream 8/9**. Quick all-in-one deployment.

Step 1 – RHEL System Preparation

```
# RHEL 9 - Register with Red Hat (skip on CentOS Stream)
sudo subscription-manager register --username=<rhel_user> --password=<rhel_pass>
sudo subscription-manager attach --auto

# Enable required repos
sudo subscription-manager repos \
    --enable=rhel-9-for-x86_64-baseos-rpms \
    --enable=rhel-9-for-x86_64-appstream-rpms \
    --enable=openstack-17-for-rhel-9-x86_64-rpms

# Update system
sudo dnf update -y

# Disable NetworkManager management of interfaces (optional for provider net)
sudo systemctl disable --now firewalld
sudo systemctl disable --now NetworkManager
sudo systemctl enable --now network

# Set hostname
sudo hostnamectl set-hostname controller.example.com
```

Step 2 – Install Packstack

```
# On RHEL 9 / CentOS Stream 9
sudo dnf install -y centos-release-openstack-caracal # CentOS Stream
# OR for RHEL:
sudo dnf install -y openstack-packstack

sudo dnf update -y
```

Step 3 – Generate & Edit Answer File

```
# Generate answer file
packstack --gen-answer-file=/root/packstack-answers.txt

# Key settings to customize
sudo sed -i \
-e "s/CONFIG_DEFAULT_PASSWORD=.*CONFIG_DEFAULT_PASSWORD=Admin1234!/\" \
-e "s/CONFIG_KEYSTONE_ADMIN_PW=.*CONFIG_KEYSTONE_ADMIN_PW=Admin1234!/\" \
-e "s/CONFIG_CONTROLLER_HOST=.*CONFIG_CONTROLLER_HOST=192.168.1.10/\" \
-e "s/CONFIG_COMPUTE_HOSTS=.*CONFIG_COMPUTE_HOSTS=192.168.1.11/\" \
-e "s/CONFIG_NETWORK_HOSTS=.*CONFIG_NETWORK_HOSTS=192.168.1.10/\" \
-e "s/CONFIG_STORAGE_HOST=.*CONFIG_STORAGE_HOST=192.168.1.10/\" \
-e "s/CONFIG_HORIZON_INSTALL=.*CONFIG_HORIZON_INSTALL=y/\" \
-e "s/CONFIG_CINDER_INSTALL=.*CONFIG_CINDER_INSTALL=y/\" \
-e "s/CONFIG_HEAT_INSTALL=.*CONFIG_HEAT_INSTALL=y/\" \
-e "s/CONFIG_NEUTRON_ML2_TYPE_DRIVERS=.*CONFIG_NEUTRON_ML2_TYPE_DRIVERS=vxlan,
/root/packstack-answers.txt
```

Step 4 – Run Packstack

```
packstack --answer-file=/root/packstack-answers.txt

# All-in-one quick install alternative:
packstack --allinone \
--default-password=Admin1234! \
--os-neutron-ml2-type-drivers=vxlan,flat
```

Step 5 – Post-Install Verification

```
source /root/keystonerc_admin
openstack service list
openstack endpoint list
```

```
openstack compute service list
openstack network agent list
```

RHEL: TripleO / Director Deployment

Production-grade RHEL OpenStack deployment using TripleO (OpenStack on OpenStack).

Step 1 – Undercloud Preparation

```
# On the director/undercloud node
sudo useradd stack
echo 'stack ALL=(ALL) NOPASSWD:ALL' | sudo tee /etc/sudoers.d/stack
sudo su - stack

# Configure hostname
sudo hostnamectl set-hostname undercloud.example.com
sudo hostnamectl set-hostname --transient undercloud.example.com
echo "192.168.1.5 undercloud.example.com undercloud" | sudo tee -a /etc/hosts

# RHEL subscription
sudo subscription-manager register
sudo subscription-manager repos \
    --enable=rhel-9-for-x86_64-baseos-rpms \
    --enable=rhel-9-for-x86_64-appstream-rpms \
    --enable=openstack-17.1-for-rhel-9-x86_64-rpms

sudo dnf update -y
sudo dnf install -y python3-tripleoclient
```

Step 2 – Configure Undercloud

```
# As stack user
cp /usr/share/python-tripleoclient/undercloud.conf.sample ~/undercloud.conf

cat > ~/undercloud.conf << 'EOF'
[DEFAULT]
undercloud_hostname = undercloud.example.com
local_ip = 192.168.24.1/24
undercloud_public_host = 192.168.24.2
undercloud_admin_host = 192.168.24.3
local_interface = eth1
```

```
local_mtu = 1500
generate_service_certificate = false
hieradata_override = /home/stack/hieradata_override.yaml

[auth]
EOF
```

Step 3 – Install Undercloud

```
openstack undercloud install
# Takes ~45 min

source ~/stackrc
openstack baremetal node list
```

Step 4 – Import & Introspect Overcloud Nodes

```
# Create instackenv.json with node details
cat > ~/instackenv.json << 'EOF'
{
  "nodes": [
    {
      "name": "control-0",
      "driver": "ipmi",
      "driver_info": {
        "ipmi_address": "192.168.1.201",
        "ipmi_username": "admin",
        "ipmi_password": "password"
      },
      "properties": {
        "capabilities": "profile:control,boot_option:local"
      }
    },
    {
      "name": "compute-0",
      "driver": "ipmi",
      "driver_info": {
        "ipmi_address": "192.168.1.202",
        "ipmi_username": "admin",
        "ipmi_password": "password"
      },
      "properties": {
        "capabilities": "profile:compute,boot_option:local"
      }
    }
  ]
}
```

```

    }
  ]
}
EOF

# Import nodes
openstack overcloud node import ~/instackenv.json

# Introspect all nodes
openstack overcloud node introspect --all-manageable --provide

```

Step 5 – Deploy Overcloud

```

# Create overcloud deploy script
cat > ~/deploy-overcloud.sh << 'EOF'
#!/bin/bash
source ~/stackrc

openstack overcloud deploy \
  --templates /usr/share/openstack-tripleo-heat-templates \
  --stack overcloud \
  --control-scale 3 \
  --compute-scale 2 \
  -e /usr/share/openstack-tripleo-heat-templates/environments/network-isolation.y
  -e /usr/share/openstack-tripleo-heat-templates/environments/storage-environment
  -e ~/templates/network-environment.yaml \
  -e ~/templates/storage-environment.yaml \
  --ntp-server pool.ntp.org \
  --timeout 90
EOF

chmod +x ~/deploy-overcloud.sh
~/deploy-overcloud.sh

```

Part 2 – RHOSO (OpenStack on OpenShift)

RHOSO 18 runs OpenStack control-plane services as pods on OpenShift (OCP 4.14+). Compute nodes are managed as External Data Plane Management (EDPM) nodes via Ansible.

Prerequisites & OpenShift Cluster

1. Install OpenShift (if not already available)

```
# Download openshift-install
wget https://mirror.openshift.com/pub/openshift-v4/clients/ocp/latest/openshift-i
tar xvf openshift-install-linux.tar.gz
sudo mv openshift-install /usr/local/bin/

# Download oc CLI
wget https://mirror.openshift.com/pub/openshift-v4/clients/ocp/latest/openshift-c
tar xvf openshift-client-linux.tar.gz
sudo mv oc kubectl /usr/local/bin/

# Create install-config.yaml
mkdir ~/ocp-install && cd ~/ocp-install

cat > install-config.yaml << 'EOF'
apiVersion: v1
baseDomain: example.com
metadata:
  name: openstack-cluster
compute:
- hyperthreading: Enabled
  name: worker
  replicas: 3
controlPlane:
  hyperthreading: Enabled
  name: master
  replicas: 3
networking:
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
  networkType: OVNKubernetes
  serviceNetwork:
  - 172.30.0.0/16
platform:
  baremetal:
    apiVIPs:
    - 192.168.1.5
    ingressVIPs:
    - 192.168.1.6
pullSecret: '{"auths": ...}' # paste your pull secret from cloud.redhat.com
sshKey: 'ssh-rsa ...' # paste your SSH public key
EOF

# Deploy cluster
```

```
openshift-install create cluster --dir ~/ocp-install --log-level=info
```

2. Configure oc Access

```
export KUBECONFIG=~/.ocp-install/auth/kubeconfig
oc whoami
oc get nodes
```

3. Prepare Storage (ODF or NFS)

```
# Install OpenShift Data Foundation (ODF) for persistent storage
# Or use NFS provisioner for lab environments

# NFS StorageClass (lab only)
oc apply -f https://raw.githubusercontent.com/kubernetes-sigs/nfs-subdir-external

cat << 'EOF' | oc apply -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-storage
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: k8s-sigs.io/nfs-subdir-external-provisioner
parameters:
  server: 192.168.1.50
  path: /exports/openstack
EOF
```

Install OpenStack Operators

1. Create openstack Namespace

```
oc new-project openstack
```

2. Create OperatorGroup and Subscription

```
cat << 'EOF' | oc apply -f -
apiVersion: v1
```

```

kind: Namespace
metadata:
  labels:
    openshift.io/cluster-monitoring: "true"
  name: openstack-operators
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openstack-operatorgroup
  namespace: openstack-operators
spec:
  targetNamespaces:
  - openstack-operators
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: openstack-operator
  namespace: openstack-operators
spec:
  channel: stable-v1.0
  installPlanApproval: Automatic
  name: openstack-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
EOF

```

3. Verify Operators Are Ready

```

# Wait for all operators to become ready
oc get csv -n openstack-operators

# Watch until all CSVs show Succeeded
watch oc get csv -n openstack-operators

# Check all CRDs are registered
oc get crd | grep openstack
oc get crd | grep rabbitmq
oc get crd | grep galera
oc get crd | grep memcached

```

4. Install Supporting Operators

```
# RabbitMQ Cluster Operator
cat << 'EOF' | oc apply -f -
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: rabbitmq-cluster-operator
  namespace: openstack-operators
spec:
  channel: latest
  installPlanApproval: Automatic
  name: rabbitmq-cluster-operator
  source: community-operators
  sourceNamespace: openshift-marketplace
EOF
```

```
# Cert-Manager (required for TLS)
cat << 'EOF' | oc apply -f -
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: cert-manager-operator-rhel9
  namespace: cert-manager-operator
spec:
  channel: stable-v1
  installPlanApproval: Automatic
  name: cert-manager-operator-rhel9
  source: redhat-operators
  sourceNamespace: openshift-marketplace
EOF
```

```
# Wait for cert-manager
oc wait --for=condition=Ready pods --all -n cert-manager --timeout=300s
```

Deploy OpenStack Control Plane

1. Create Secret for Service Passwords

```
cat << 'EOF' | oc apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: osp-secret
  namespace: openstack
```

```
type: Opaque
stringData:
  AdminPassword: Admin1234!
  DbRootPassword: DbRoot1234!
  DatabasePassword: Database1234!
  KeystoneAdminPassword: Admin1234!
  GlancePassword: Glance1234!
  NovaPassword: Nova1234!
  NeutronPassword: Neutron1234!
  CinderPassword: Cinder1234!
  HeatPassword: Heat1234!
  HorizonSecret: HorizonSecret123
  MetadataSecret: MetaSecret123
  RabbitMQPassword: Rabbit1234!
  MemcacheSecret: MemcacheSecret123
EOF
```

2. Create OpenStackControlPlane CR

```
cat << 'EOF' | oc apply -f -
apiVersion: core.openstack.org/v1beta1
kind: OpenStackControlPlane
metadata:
  name: openstack-galera-network-isolation
  namespace: openstack
spec:
  secret: osp-secret
  storageClass: nfs-storage

  # Galera (MariaDB cluster)
  galera:
    enabled: true
    templates:
      openstack:
        storageRequest: 500M
        secret: osp-secret
        replicas: 3

  # RabbitMQ
  rabbitmq:
    enabled: true
    templates:
      rabbitmq:
        replicas: 3
```

```
# Memcached
memcached:
  enabled: true
  templates:
    memcached:
      replicas: 3

# Keystone
keystone:
  enabled: true
  template:
    databaseInstance: openstack
    secret: osp-secret

# Glance
glance:
  enabled: true
  template:
    databaseInstance: openstack
    storageClass: nfs-storage
    storageRequest: 10G
    secret: osp-secret
    glanceAPIs:
      default:
        replicas: 1
        type: single

# Cinder
cinder:
  enabled: true
  template:
    databaseInstance: openstack
    secret: osp-secret
    cinderAPI:
      replicas: 1
    cinderScheduler:
      replicas: 1
    cinderBackup:
      replicas: 0 # Set to 1 with backend configured
    cinderVolumes:
      volume1:
        replicas: 1

# Nova (Compute API)
nova:
  enabled: true
  template:
```

```
    secret: osp-secret
    apiDatabaseInstance: openstack
    cellDatabaseInstance: openstack

# Neutron
neutron:
    enabled: true
    template:
        databaseInstance: openstack
        secret: osp-secret

# Horizon
horizon:
    enabled: true
    template:
        replicas: 1
        secret: osp-secret

# Heat
heat:
    enabled: true
    template:
        databaseInstance: openstack
        secret: osp-secret

# Placement
placement:
    enabled: true
    template:
        databaseInstance: openstack
        secret: osp-secret

# OVN
ovn:
    enabled: true
    template:
        ovnDBCluster:
            ovndbcluster-nb:
                storageRequest: 10G
                dbType: NB
                networkAttachment: internalapi
            ovndbcluster-sb:
                storageRequest: 10G
                dbType: SB
                networkAttachment: internalapi
        ovnNorthd:
            replicas: 1
```

```

    ovnController:
      networkAttachment: tenant
      nodeSelector:
        node-role.kubernetes.io/worker: ""
EOF

```

3. Monitor Control Plane Deployment

```

# Watch overall status
watch oc get OpenStackControlPlane -n openstack

# Check individual service pods
oc get pods -n openstack

# Watch until ready
oc wait --for=condition=Ready OpenStackControlPlane/openstack-galera-network-isol
  -n openstack --timeout=600s

# Verify services
oc get pods -n openstack | grep -E "keystone|glance|nova|neutron|cinder|horizon"

```

4. Extract Admin Credentials

```

# Get admin password
oc get secret osp-secret -n openstack -o jsonpath='{.data.AdminPassword}' | base64

# Get Keystone endpoint
oc get route keystone-public -n openstack -o jsonpath='{.spec.host}'

# Create local openrc
KEYSTONE_URL=$(oc get route keystone-public -n openstack -o jsonpath='{.spec.host}
cat > ~/openstack-openrc.sh << EOF
export OS_AUTH_URL=https://${KEYSTONE_URL}/v3
export OS_USERNAME=admin
export OS_PASSWORD=Admin1234!
export OS_PROJECT_NAME=admin
export OS_USER_DOMAIN_NAME=Default
export OS_PROJECT_DOMAIN_NAME=Default
export OS_IDENTITY_API_VERSION=3
export OS_CACERT=/etc/pki/tls/certs/ca-bundle.crt
EOF

source ~/openstack-openrc.sh
openstack service list

```

Deploy OpenStack Data Plane (Compute Nodes)

1. Prepare EDPM (Compute) Nodes

```
# On each compute node (RHEL 9)
sudo subscription-manager register
sudo subscription-manager repos \
  --enable=rhel-9-for-x86_64-baseos-rpms \
  --enable=rhel-9-for-x86_64-appstream-rpms

sudo dnf update -y

# Ensure SSH key access from the OCP cluster
# Copy the public key from the dataplane SSH secret
```

2. Create SSH Key Secret for EDPM

```
# Generate SSH keypair
ssh-keygen -t rsa -b 4096 -f ~/dataplane-ssh-key -N ""

# Create secret
oc create secret generic dataplane-ansible-ssh-private-key-secret \
  --from-file=authorized_keys=~/dataplane-ssh-key.pub \
  --from-file=ssh-privatekey=~/dataplane-ssh-key \
  --from-file=ssh-publickey=~/dataplane-ssh-key.pub \
  -n openstack
```

3. Create OpenStackDataPlaneNodeSet

```
cat << 'EOF' | oc apply -f -
apiVersion: dataplane.openstack.org/v1beta1
kind: OpenStackDataPlaneNodeSet
metadata:
  name: openstack-edpm
  namespace: openstack
spec:
  # Deploy strategy
  preProvisioned: true

  # Shared service configurations
```

```

services:
- download-cache
- bootstrap
- configure-network
- validate-network
- install-os
- configure-os
- run-os
- install-certs
- ovn
- neutron-metadata
- libvirt
- nova
- telemetry

nodeTemplate:
  # Ansible configuration
  ansible:
    ansibleUser: cloud-admin
    ansiblePort: 22
    ansibleVars:
      service_net_map:
        nova_api_network: internalapi
        neutron_tenant_network: tenant
      edpm_network_config_template: |
        ---
        {% set mtu_list = [ctlplane_mtu] %}
        {% for network in role_networks %}
        {{ mtu_list.append(lookup('vars', networks_lower[network] ~ '_mtu')) }}
        {%- endfor %}
        {% set min_viable_mtu = mtu_list | max %}
        network_config:
          - type: ovs_bridge
            name: br-ctlplane
            mtu: {{ min_viable_mtu }}
            use_dhcp: false
            dns_servers: {{ ctlplane_dns_nameservers }}
            addresses:
              - ip_netmask: {{ ctlplane_ip }}/{{ ctlplane_cidr }}
            routes: {{ ctlplane_host_routes }}
            members:
              - type: interface
                name: nic1
                mtu: {{ min_viable_mtu }}
                primary: true
      edpm_ovn_bridge_mappings:
        - datacentre:br-ex

```

```

    edpm_ovn_bridge: br-int
    edpm_ovn_encap_type: geneve
    edpm_iscsid_image: "{{ registry_url }}/rhosp-rhel9/openstack-iscsid:lates
    edpm_logrotate_crond_image: "{{ registry_url }}/rhosp-rhel9/openstack-cro
    edpm_nova_compute_container_image: "{{ registry_url }}/rhosp-rhel9/openst
    edpm_libvirt_image: "{{ registry_url }}/rhosp-rhel9/openstack-libvirt:lat
    timesync_ntp_servers:
    - hostname: pool.ntp.org

# SSH access
networkConfig: {}
managementNetwork: ctlplane
ansible:
    ansibleSSHPrivateKeySecret: dataplane-ansible-ssh-private-key-secret

# Node definitions
nodes:
    edpm-compute-0:
        hostName: edpm-compute-0
        ansible:
            ansibleHost: 192.168.1.20
        networks:
            - name: ctlplane
              subnetName: subnet1
              defaultRoute: true
              fixedIP: 192.168.1.20
            - name: internalapi
              subnetName: subnet1
            - name: storage
              subnetName: subnet1
            - name: tenant
              subnetName: subnet1

    edpm-compute-1:
        hostName: edpm-compute-1
        ansible:
            ansibleHost: 192.168.1.21
        networks:
            - name: ctlplane
              subnetName: subnet1
              defaultRoute: true
              fixedIP: 192.168.1.21
            - name: internalapi
              subnetName: subnet1
            - name: storage
              subnetName: subnet1
            - name: tenant

```

```
        subnetName: subnet1
EOF
```

4. Create OpenStackDataPlaneDeployment

```
cat << 'EOF' | oc apply -f -
apiVersion: dataplane.openstack.org/v1beta1
kind: OpenStackDataPlaneDeployment
metadata:
  name: openstack-edpm-deployment
  namespace: openstack
spec:
  nodeSets:
  - openstack-edpm
EOF
```

5. Monitor Data Plane Deployment

```
# Watch deployment progress
oc get OpenStackDataPlaneDeployment -n openstack
watch oc get OpenStackDataPlaneDeployment -n openstack

# Watch Ansible jobs
oc get jobs -n openstack

# Get Ansible job logs
oc logs -l app=edpm-ansible -n openstack -f

# Wait for completion
oc wait --for=condition=Ready \
  OpenStackDataPlaneDeployment/openstack-edpm-deployment \
  -n openstack --timeout=3600s
```

Post-Deployment Validation

RHOSO Validation

```
source ~/openstack-openrc.sh

# Service and endpoint check
openstack service list
```

```
openstack endpoint list
openstack compute service list
openstack network agent list
openstack volume service list

# Create test resources
openstack project create --description "Test Project" test-project
openstack user create --project test-project --password Test1234! test-user
openstack role add --project test-project --user test-user member

# Create network
openstack network create --external \
    --provider-physical-network datacentre \
    --provider-network-type flat \
    external-network

openstack subnet create \
    --network external-network \
    --dns-nameserver 8.8.8.8 \
    --gateway 10.0.0.1 \
    --allocation-pool start=10.0.0.100,end=10.0.0.200 \
    --subnet-range 10.0.0.0/24 \
    external-subnet

openstack network create tenant-network
openstack subnet create \
    --network tenant-network \
    --subnet-range 192.168.100.0/24 \
    tenant-subnet

openstack router create test-router
openstack router set test-router --external-gateway external-network
openstack router add subnet test-router tenant-subnet

# Create and boot test instance
openstack flavor create --ram 512 --vcpus 1 --disk 1 m1.tiny
openstack keypair create --public-key ~/.ssh/id_rsa.pub mykey

openstack server create \
    --image cirros \
    --flavor m1.tiny \
    --network tenant-network \
    --key-name mykey \
    test-instance

# Wait and check
openstack server list
```

```
openstack server show test-instance

# Assign floating IP
openstack floating ip create external-network
openstack server add floating ip test-instance <floating_ip>

# Test SSH
ssh -i ~/.ssh/id_rsa cirros@<floating_ip>
```

Common Post-Deployment Tasks

Security Groups

```
source ~/admin-openrc.sh    # or ~/openstack-openrc.sh for RHOSO

# Create security group
openstack security group create web-sg --description "Web Security Group"

# Allow SSH
openstack security group rule create \
    --protocol tcp --dst-port 22 \
    --remote-ip 0.0.0.0/0 web-sg

# Allow ICMP (ping)
openstack security group rule create \
    --protocol icmp web-sg

# Allow HTTP/HTTPS
openstack security group rule create --protocol tcp --dst-port 80 web-sg
openstack security group rule create --protocol tcp --dst-port 443 web-sg
```

Create Flavors

```
openstack flavor create m1.tiny    --ram 512  --vcpus 1  --disk 1
openstack flavor create m1.small   --ram 2048 --vcpus 1  --disk 20
openstack flavor create m1.medium  --ram 4096 --vcpus 2  --disk 40
openstack flavor create m1.large   --ram 8192 --vcpus 4  --disk 80
```

Upload Additional Images

```
# Ubuntu 22.04
```

```
wget https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.im
openstack image create "Ubuntu-22.04" \
  --file jammy-server-cloudimg-amd64.img \
  --disk-format qcow2 --container-format bare \
  --public

# RHEL 9 (requires valid subscription image)
openstack image create "RHEL-9" \
  --file rhel-9-x86_64-kvm.qcow2 \
  --disk-format qcow2 --container-format bare \
  --public
```

Cinder Volume Management

```
# Create volume
openstack volume create --size 10 my-volume

# Attach to instance
openstack server add volume test-instance my-volume

# Create volume snapshot
openstack volume snapshot create --volume my-volume my-snapshot

# Create volume from image
openstack volume create --image cirros --size 5 boot-volume
```

Troubleshooting

Classic OpenStack

```
# Check service logs
sudo journalctl -u nova-api -f
sudo journalctl -u neutron-server -f
sudo journalctl -u keystone -f

# Test Keystone manually
curl -s -o /dev/null -w "%{http_code}" http://192.168.1.10:5000/v3

# Check RabbitMQ queues
sudo rabbitmqctl list_queues

# Nova placement check
```

```
openstack resource provider list
openstack resource class list

# Neutron agent check
openstack network agent list
neutron ext-list

# Database connections
mysql -u nova -pNovaDB123 nova -e "show tables;"
```

RHOSO / OCP

```
# Check OpenStackControlPlane status
oc describe OpenStackControlPlane -n openstack

# Check failing pods
oc get pods -n openstack | grep -v Running
oc describe pod <pod-name> -n openstack
oc logs <pod-name> -n openstack

# Check operator logs
oc logs -l control-plane=controller-manager -n openstack-operators

# Check Galera / DB status
oc exec -it openstack-galera-0 -n openstack -- mysql -u root -e "SHOW STATUS LIKE

# Check RabbitMQ
oc exec -it rabbitmq-server-0 -n openstack -- rabbitmqctl cluster_status

# Check OVN
oc exec -it ovn-controller-<pod> -n openstack -- ovs-vsctl show

# DataPlane deployment logs
oc get OpenStackDataPlaneDeployment -n openstack
oc describe OpenStackDataPlaneDeployment openstack-edpm-deployment -n openstack
oc logs -l job-name=openstack-edpm-deployment-configure-network -n openstack

# Re-run failed deployment
oc delete OpenStackDataPlaneDeployment openstack-edpm-deployment -n openstack
# Then re-apply the deployment YAML
```

GitHub Repository Structure

Use the following structure to organize this content on GitHub:

```
openstack-rhoso-deployment/
├── README.md                                # This file
├── classic-openstack/
│   ├── ubuntu/
│   │   ├── devstack/
│   │   │   ├── local.conf                # DevStack config
│   │   │   └── deploy.sh                 # Automated deploy script
│   │   └── manual/
│   │       ├── 01-base-setup.sh
│   │       ├── 02-keystone.sh
│   │       ├── 03-glance.sh
│   │       ├── 04-nova-controller.sh
│   │       ├── 05-neutron-controller.sh
│   │       ├── 06-cinder.sh
│   │       ├── 07-horizon.sh
│   │       ├── 08-nova-compute.sh
│   │       └── admin-openrc.sh.template
│   └── rhel/
│       ├── packstack/
│       │   ├── packstack-answers.txt.template
│       │   └── deploy.sh
│       └── tripleo/
│           ├── instackenv.json.template
│           ├── undercloud.conf.template
│           ├── deploy-overcloud.sh
│           └── templates/
│               ├── network-environment.yaml
│               └── storage-environment.yaml
└── rhoso/
    ├── operators/
    │   ├── namespace.yaml
    │   ├── operatorgroup.yaml
    │   └── subscription.yaml
    ├── controlplane/
    │   ├── secret.yaml
    │   ├── controlplane.yaml
    │   └── network-attachment-definitions.yaml
    ├── dataplane/
    │   ├── ssh-secret.yaml
    │   ├── nodeset.yaml
    │   └── deployment.yaml
    └── validation/
        └── post-deploy-checks.sh
```

Quick GitHub Commands

```
# Initialize repo
git init openstack-rhoso-deployment
cd openstack-rhoso-deployment

git init
git add .
git commit -m "Initial commit: OpenStack & RHOSO deployment guide"

# Add remote (replace with your GitHub repo URL)
git remote add origin https://github.com/<your-username>/openstack-rhoso-deployme

# Push to GitHub
git branch -M main
git push -u origin main

# Tag a release
git tag -a v1.0.0 -m "Initial release - OpenStack 2024.1 / RHOSO 18"
git push origin v1.0.0
```

References

- [OpenStack Documentation](#)
- [RHOSO 18 Documentation](#)
- [OpenShift 4 Documentation](#)
- [DevStack Documentation](#)
- [TripleO Documentation](#)
- [EDPM \(Data Plane\) Docs](#)

Generated for OpenStack 2024.1 (Caracal) / RHOSO 18 / OCP 4.14+